

S7. Redis Streams CLI 실습

At-least-once 재처리 시뮬레이션

XADD · XGROUP · XREADGROUP · XACK · ~~미~~ACK 재수신 · Consumer 2개 분배

2026.05 | M2 S7

S6 핵심 복습 — 오늘 실습의 기반

S6에서 배운 이론 — 오늘 직접 타이핑으로 확인

Streams = 로그

추가 전용 자료구조
XADD로 적재

Consumer Group

여러 Consumer
중복 없이 분배

PEL

읽혔지만 XACK 없는
메시지 목록

XACK

처리 완료 선언
PEL 제거

At-least-once

PEL + XACK
재시작 시 재수신

이론 → 실습. 손으로 타이핑해야 진짜 이해한 것.

S7 실습 개요 — 5단계

오늘 실습 목표: 직접 타이핑으로 At-least-once 재처리 시뮬레이션

단계 1	환경 준비	Redis 기동 + CLI 접속 확인	10분
단계 2	Streams 기초	XADD → XLEN → XRANGE 조회	10분
단계 3	Consumer Group	XGROUP CREATE → XREADGROUP → XACK	15분
단계 4	미ACK 재수신	XACK 전 종료 → 재시작 → 재수신 확인	15분
단계 5	Consumer 2개	2 Consumer 동시 실행 → 메시지 분배 확인	10분

환경 준비 — Redis CLI 접속

터미널을 열고 Redis에 접속한다

Redis 기동 확인

```
docker ps | grep redis
```

redis 컨테이너 실행 중 확인

Redis CLI 접속

```
docker exec -it <container> redis-cli
```

또는 redis-cli (로컬 설치 시)

접속 확인

```
PING
```

응답: PONG

버전 확인

```
INFO server | grep redis_version
```

6.x 이상 권장

PONG 응답 확인 = 실습 준비 완료

실습 시작 전 — 깨끗한 상태 만들기

스트림 삭제	DEL eventStream	기존 스트림 제거
키 목록 확인	KEYS *	남은 키 없는지 확인
플러시 (주의)	FLUSHDB	현재 DB 전체 삭제 (팀 환경에서는 금지)

이전 실습 데이터가 있으면 충돌 발생 — 먼저 클린업

⚠️ 공용 Redis 사용 시 DEL eventStream만 사용. FLUSHDB는 전체 데이터 삭제.

실습 1 — XADD: 이벤트 스트림에 항목 추가

XADD eventStream * field value — 이벤트 항목 추가

XADD eventStream * eventType NFT_ISSUED tokenId 1 txHash 0xaaa111 recipient 0xuser1

XADD eventStream * eventType NFT_ISSUED tokenId 2 txHash 0xaaa222 recipient 0xuser2

XADD eventStream * eventType NFT_BURNED tokenId 1 txHash 0xbbb111 recipient 0xuser1

XADD eventStream * eventType NFT_ISSUED tokenId 3 txHash 0xaaa333 recipient 0xuser3

XADD eventStream * eventType TRANSFER tokenId 2 txHash 0xccc111 recipient 0xuser3

반환값: "1700000001234-0" 형식의 Entry ID — 이것이 메시지 추적 키

XADD 결과 확인 — 반환값 분석

각 XADD 명령 후 반환되는 Entry ID 구조 확인

예시 반환: "1748000001234-0"

1748000001234
Unix 타임스탬프(ms)

명령 실행 시각

-
구분자

0
seq 번호

동일 ms 내 순서 (0부터 시작)

같은 ms에 여러 XADD 시: 1748000001234-0, 1748000001234-1, 1748000001234-2...

이 ID가 나중에 XACK, XPENDING에서 메시지 식별자로 사용됨 — 기억해두기

실습 1 — XLEN / XRANGE: 스트림 내용 확인

적재한 항목 수와 내용 확인

XLEN eventStream

스트림 항목 수 확인

→ 5 (5개 추가했으므로)

XRANGE eventStream - +

전체 항목 조회 (처음~끝)

→ 5개 항목 내용 출력

XRANGE eventStream - + COUNT 2

처음 2개만 조회

→ 2개 항목 출력

XREVRANGE eventStream + -

역순 조회 (최신→오래된)

→ 최신 항목부터 출력

XRANGE - + = 처음(-) 부터 끝(+) 까지. 특수 심볼 외우기.

XRANGE 출력 해석 — 무엇이 보이는가

XRANGE 출력 구조 이해

1748000001234-0

- 1) "eventType"
- 2) "NFT_ISSUED"
- 3) "tokenId"
- 4) "1"
- 5) "txHash"
- 6) "0xaaa111"
- 7) "recipient"
- 8) "0xuser1"

첫 줄 = Entry ID (타임스탬프-seq) / 이후 = field-value 쌍 (짝수 개)

field-value 쌍: XADD에서 지정한 순서 그대로 출력됨

실습 포인트 — XADD 5개 후 상태

5개 항목 추가 완료 후 확인 체크리스트

✓	XLEN = 5	5개가 정확히 추가됨
✓	XRANGE - + 출력에 5개 항목 보임	각 항목에 eventType, txHash 포함
✓	Entry ID 형식: timestamp-seq	1748...-0 형식 확인
✓	XREVRANGE 출력은 역순	마지막에 추가한 항목이 첫 번째로 나옴

4가지 모두 확인 완료 = 단계 2 완료

실습 2 — XGROUP CREATE: Consumer Group 생성

Consumer Group 생성 + 스트림 초기화

그룹 생성

```
XGROUP CREATE eventStream nftConsumers 0 MKSTREAM
```

0 = 처음부터 (모든 기존 메시지 대상)

그룹 확인

```
XINFO GROUPS eventStream
```

생성된 그룹 정보 확인

Consumer 목록

```
XINFO CONSUMERS eventStream nftConsumers
```

현재 Consumer 없음 확인

⚠ 그룹 이미 존재 시: BUSYGROUP 에러 → XGROUP DESTROY 후 재생성 (DEL eventStream 권장)

XINFO GROUPS 출력 해석

그룹 정보 확인 — 각 필드 의미

- 1) "name"
- 2) "nftConsumers"
- 3) "consumers"
- 4) (integer) 0
- 5) "pending"
- 6) (integer) 0
- 7) "last-delivered-id"
- 8) "0-0"
- 9) "lag"
- 10) (integer) 5

consumers

현재 Consumer 수 (0 = 아직 없음)

pending

PEL에 있는 메시지 수 (0 = 정상)

last-delivered-id

마지막으로 배달된 Entry ID

lag

아직 배달 안 된 메시지 수 (5 = 5개 대기 중)

lag = 5 → 아직 어떤 Consumer도 읽지 않은 5개 메시지 대기 중

실습 2 — XREADGROUP: 메시지 읽기

Consumer Group으로 메시지 읽기

첫 번째 읽기	XREADGROUP GROUP nftConsumers consumer-1 COUNT 2 STREAMS eventStream >	2개 읽기
PEL 확인	XPENDING eventStream nftConsumers - + 10	2개가 PEL에 등록됨
두 번째 읽기	XREADGROUP GROUP nftConsumers consumer-1 COUNT 3 STREAMS eventStream >	나머지 3개 읽기
PEL 재 확인	XPENDING eventStream nftConsumers - + 10	총 5개 PEL에 있음

> = 미처리 새 메시지. 5개 읽으면 다시 > 호출 시 빈 결과 반환.

XPENDING 출력 분석 — PEL 내용 이해

XPENDING 출력: 메시지별 상세 정보

```
1) 1748000001234-0
   consumer-1
   (integer) 1500 ← 마지막 배달 후 경과 ms
   (integer) 1    ← 배달 횟수 (delivery_count)
2) 1748000001235-0
   consumer-1
   ...
(5개 항목 출력)
```

delivery_count = 1 → 처음 배달됨. 재배달 시 증가. ≥3이면 DLQ 이동 기준(S11)

이 5개 메시지는 XACK를 받을 때까지 영원히 PEL에 잔류한다

실습 2 — XACK: 처리 완료 선언

XREADGROUP으로 읽은 Entry ID로 XACK 전송

⚠ 아래 ID는 예시. 실제 실습에서는 XPENDING 출력의 ID 복사해서 사용

XACK eventStream nftConsumers 1748000001234-0

→ 1 (ACK된 수)

XPENDING eventStream nftConsumers - + 10

→ 4개로 줄어듦

XACK eventStream nftConsumers 1748000001235-0 1748000001236-0

→ 2 (2개 동시 ACK)

XACK eventStream nftConsumers [나머지 ID들]

→ 전부 ACK

XPENDING eventStream nftConsumers - + 10

→ (empty) PEL 비어있음

PEL 비어있음 = 모든 메시지 처리 완료 ✓

XACK 전후 PEL 상태 비교

XACK 전 — PEL에 5개 잔류

Entry 1: consumer-1 | delivery_count=1

Entry 2: consumer-1 | delivery_count=1

Entry 3: consumer-1 | delivery_count=1

Entry 4: consumer-1 | delivery_count=1

Entry 5: consumer-1 | delivery_count=1

XACK 후 — PEL 비어있음

(empty array)
모든 메시지 XACK 완료

XINFO GROUPS에서 pending = 0 확인

PEL = 0 = 처리 완전 완료. XACK 없이 종료 시 PEL에 계속 남아있음.

XRANGE — XACK 후에도 메시지는 남아있다

XRANGE eventStream - + → 여전히 5개 항목 출력

XACK는 Consumer Group의 PEL에서만 제거. 스트림 자체 데이터는 그대로 보존.

PEL	XACK 후 → 사라짐	Consumer Group 관점 완료 표시
Stream	XACK 후 → 유지됨	감사 추적 + 다른 그룹 소비 가능
XDEL	명시적 삭제 명령	Stream에서 실제 삭제 (사용 주의)

XACK = 처리 완료 표시. 스트림에서 삭제되는 것이 아니다.

금융 감사 목적: 스트림 데이터는 보존. PEL만 관리하면 된다.

단계 1~2 완료 확인

지금까지 확인한 것들

✓	XADD	5개 항목 추가 + Entry ID 반환 확인
✓	XLEN	5 반환 확인
✓	XRANGE	5개 항목 내용 출력 확인
✓	XGROUP CREATE	nftConsumers 그룹 생성 확인
✓	XREADGROUP >	5개 읽기 + PEL 등록 확인
✓	XPENDING	5개 항목 PEL에 있음 확인
✓	XACK	전체 ACK 후 PEL 비어있음 확인

단계 3 — 미ACK 재수신 시뮬레이션

핵심 실습: XACK 전에 종료 → 재시작 후 동일 메시지 재수신 확인

준비

새 이벤트 3개 추가
(XADD)

읽기

XREADGROUP > 로 3개
읽기

ACK 안 함

XACK 없이 그대로 둠
(프로세스 종료
시뮬레이션)

재시작

새 Consumer로
XREADGROUP 0
(PEL 재수신)

확인

동일 3개 메시지 다시
수신됨

이 시나리오 = At-least-once 동작의 직접 확인

미ACK 실습 1단계 — 새 이벤트 3개 추가

기존 스트림 초기화 후 새 이벤트 3개 추가

스트림 재생성	DEL eventStream
그룹 재생성	XGROUP CREATE eventStream nftConsumers 0 MKSTREAM
이벤트 1 추가	XADD eventStream * eventType NFT_ISSUED tokenId 10 txHash 0xdd001
이벤트 2 추가	XADD eventStream * eventType NFT_ISSUED tokenId 11 txHash 0xdd002
이벤트 3 추가	XADD eventStream * eventType NFT_BURNED tokenId 10 txHash 0xdd003
확인	XLEN eventStream → 3

3개 이벤트 준비 완료

미ACK 실습 2단계 — 읽기 + XACK 하지 않기

consumer-1이 3개 읽기 → ACK 없이 종료 시뮬레이션

```
XREADGROUP GROUP nftConsumers consumer-1 COUNT 3 STREAMS eventStream >
```

→ 3개 메시지 반환됨 (Entry ID 메모해두기)

❌ XACK 하지 말 것 — 이것이 프로세스 크래시 시뮬레이션

```
XPENDING eventStream nftConsumers - + 10
```

→ 3개가 PEL에 있음 (consumer-1, delivery_count=1)

이 상태 = consumer-1이 읽고 처리 중 크래시한 상황

미ACK 실습 3단계 — > 로 읽으면 빈 결과

```
XREADGROUP GROUP nftConsumers consumer-1 COUNT 10 STREAMS eventStream >
```

(empty array) ← 이미 읽은 메시지는 > 로 재수신 안 됨

이유: > 는 미처리 새 메시지만 반환. 이미 읽고 PEL에 있는 메시지는 제외.

재수신 방법: > 대신 0 사용 → PEL에 있는 미ACK 메시지 재수신

실제 복구 시나리오: 재시작된 Consumer가 0으로 XREADGROUP 호출

미ACK 실습 4단계 — 0 으로 PEL 재수신

재시작 시뮬레이션 — 0 으로 PEL 메시지 재수신

```
XREADGROUP GROUP nftConsumers consumer-1 COUNT 10 STREAMS eventStream 0
```

```
1) 1748000001234-0
  1) eventType
  2) NFT_ISSUED
  3) tokenId
  4) 10
  ... (3개 모두 반환)
```

→ 동일한 3개 메시지 다시 수신됨!

이것이 At-least-once. 크래시 후 재시작해도 메시지 유실 없음.

delivery_count: XPENDING 확인 시 1→2로 증가됨 확인

미ACK 실습 5단계 — 재수신 후 XACK 완료

재수신한 메시지 처리 완료 → XACK 전송

XACK eventStream nftConsumers <Entry ID 1> <Entry ID 2> <Entry ID 3>

→ 3 (3개 ACK 완료)

XPENDING eventStream nftConsumers - + 10

(empty array) ← PEL 비어있음. 모든 메시지 처리 완료.

At-least-once 시뮬레이션 완료 ✓
크래시 → PEL 잔류 → 재시작 → 0으로 재수신 → 처리 → XACK

delivery_count 확인 — 재배달 횟수 추적

XPENDING에서 delivery_count 확인 방법

실습: XACK 없이 XREADGROUP 0 을 3번 반복 호출 → delivery_count 증가 확인

1회차	XREADGROUP GROUP nftConsumers consumer-1 COUNT 3 STREAMS eventStream 0	delivery_count = 1
2회차	XREADGROUP GROUP nftConsumers consumer-1 COUNT 3 STREAMS eventStream 0	delivery_count = 2
3회차	XREADGROUP GROUP nftConsumers consumer-1 COUNT 3 STREAMS eventStream 0	delivery_count = 3 → DLQ 대상
확인	XPENDING eventStream nftConsumers - + 10	delivery_count 값 확인

delivery_count ≥ 3 = DLQ 이동 기준. S11에서 moveToDLQ() 구현.

미ACK 재수신 실습 — 핵심 포인트 정리

단계 3 완료 — 이것을 확인했다

읽기 후 ACK 없음

PEL에 3개 잔류 확인

> 재호출

빈 결과 — 이미 읽은 메시지 제외됨

0 으로 재호출

PEL의 3개 메시지 재수신 — At-least-once 확인

XACK 완료

PEL 비어있음 확인

delivery_count

재배달 시마다 +1 증가 확인

이 경험이 S9 '처리 순서 불변 규칙'의 기반이 됨

단계 4 — Consumer 2개 동시 실행

두 Consumer가 같은 그룹으로 읽을 때 메시지가 어떻게 분배되는가?

consumer-1 (터미널 1)

XREADGROUP > 호출
→ 일부 메시지 받음
→ 나머지 consumer-2 몫

consumer-2 (터미널 2)

XREADGROUP > 호출
→ 남은 메시지 받음
→ consumer-1 몫 없음

각 메시지는 하나의 Consumer에만 배달 — 중복 없음

실습 시뮬레이션: 터미널 창 2개 또는 순서대로 교대로 실행

Consumer 2개 실습 — 데이터 준비

스트림 초기화 + 6개 이벤트 추가

DEL eventStream

XGROUP CREATE eventStream nftConsumers 0 MKSTREAM

XADD eventStream * eventType NFT_ISSUED tokenId 1 txHash 0xee001

XADD eventStream * eventType NFT_ISSUED tokenId 2 txHash 0xee002

XADD eventStream * eventType NFT_ISSUED tokenId 3 txHash 0xee003

XADD eventStream * eventType NFT_BURNED tokenId 1 txHash 0xff001

XADD eventStream * eventType TRANSFER tokenId 2 txHash 0xff002

XADD eventStream * eventType NFT_ISSUED tokenId 4 txHash 0xee004

XLEN eventStream → 6 확인

Consumer 2개 분배 — consumer-1 먼저 읽기

터미널 1 (consumer-1): COUNT 3으로 3개 읽기

```
XREADGROUP GROUP nftConsumers consumer-1 COUNT 3 STREAMS eventStream >
```

```
1) Entry 1 (NFT_ISSUED tokenId=1)  
2) Entry 2 (NFT_ISSUED tokenId=2)  
3) Entry 3 (NFT_ISSUED tokenId=3)
```

consumer-1이 Entry 1~3 수신

XPENDING 확인: consumer-1에 3개 할당됨

```
XPENDING eventStream nftConsumers - + 10
```

→ consumer-1: Entry1, Entry2, Entry3 (delivery_count=1 각)

Consumer 2개 분배 — consumer-2 이어서 읽기

터미널 2 (consumer-2): > 로 나머지 읽기

```
XREADGROUP GROUP nftConsumers consumer-2 COUNT 10 STREAMS eventStream >
```

```
4) Entry 4 (NFT_BURNED tokenId=1)  
5) Entry 5 (TRANSFER tokenId=2)  
6) Entry 6 (NFT_ISSUED tokenId=4)
```

consumer-2가 Entry 4~6 수신 — consumer-1 것과 중복 없음

메시지 분배: consumer-1(E1~3) / consumer-2(E4~6) — 각 3개씩

```
XPENDING eventStream nftConsumers - + 10
```

→ consumer-1: 3개, consumer-2: 3개 (총 6개 PEL)

Consumer 2개 분배 결과 — 시각적 확인

6개 메시지 분배 결과

E1
NFT_ISSUED
tokenId=1

E2
NFT_ISSUED
tokenId=2

E3
NFT_ISSUED
tokenId=3

E4
NFT_BURNED
tokenId=1

E5
TRANSFER
tokenId=2

E6
NFT_ISSUED
tokenId=4

consumer-1 담당 (E1~E3)

consumer-2 담당 (E4~E6)

Redis가 자동 분배 — Consumer 코드는 XREADGROUP > 호출만 하면 됨

Consumer 하나 추가 = 처리량 증가. 코드 변경 없음. Redis가 자동 분배.

Consumer 2개 — 독립적 XACK

각 Consumer가 독립적으로 XACK 전송

터미널 1 — consumer-1 ACK

XACK eventStream nftConsumers E1 E2 E3

터미널 2 — consumer-2 ACK

XACK eventStream nftConsumers E4 E5 E6

전체 PEL 확인

XPENDING eventStream nftConsumers - + 10

(empty array) ← 모두 ACK 완료. 분산 처리 성공.

Consumer 2개 분배 + 독립 ACK 실습 완료 ✓

Consumer 수 증가 효과 — 이론 vs 현실

Consumer 수와 처리량의 관계

Consumer 1개	Consumer 2개	Consumer 4개	Consumer 10개
10 TPS	~18 TPS	~32 TPS	~50 TPS
싱글 처리 병목 발생 가능	네트워크 오버헤드 약 80% 효율	DB 연결 경쟁 약 80% 효율	DB가 병목 DB 스케일 필요

현실: Consumer 무한 증가 불가. DB 연결 풀 / Redis 처리량이 실제 상한선.

실습 추가 — XAUTOCLAIM: 오래된 미ACK 재배포분

Consumer 장애 시뮬레이션 + XAUTOCLAIM 재배포분

시나리오: consumer-1이 3개 읽고 XACK 없이 종료 → consumer-2가 XAUTOCLAIM으로 인계

1	consumer-1이 3개 읽기 (XACK 안 함)	XREADGROUP GROUP nftConsumers consumer-1 COUNT 3 STREAMS eventStream >
2	PEL idle time 확인 (장애 시뮬레이션용)	XPENDING eventStream nftConsumers - + 10
3	consumer-2가 XAUTOCLAIM (1ms 이상 미ACK)	XAUTOCLAIM eventStream nftConsumers consumer-2 1 0-0
4	consumer-2 PEL 확인	XPENDING eventStream nftConsumers - + 10

XAUTOCLAIM 후: PEL에서 consumer-1 → consumer-2로 소유권 이전됨

XAUTOCLAIM 결과 — consumer 소유권 이전

XAUTOCLAIM 후 PEL 상태 변화

XAUTOCLAIM 전

consumer-1: E1 (pending)
consumer-1: E2 (pending)
consumer-1: E3 (pending)



XAUTOCLAIM 후

consumer-2: E1 (재배분)
consumer-2: E2 (재배분)
consumer-2: E3 (재배분)

delivery_count: 1→2 증가 (재배달). consumer-1은 이 메시지 더 이상 책임 없음.

이 메커니즘이 Consumer 장애 자동 복구를 가능하게 함

XACK eventStream nftConsumers E1 E2 E3 ← consumer-2가 처리 완료

실습 추가 — MAXLEN으로 스트림 크기 관리

MAXLEN 옵션으로 오래된 항목 자동 정리

XADD eventStream MAXLEN = 5 * eventType NFT_ISSUED tokenId 100 txHash 0xzz001

→ 스트림 크기를 약 5개로 유지 (초과 시 오래된 것 자동 삭제)

MAXLEN 전	XLEN eventStream	기존 항목 수 확인
MAXLEN으로 10개 추가	XADD ... MAXLEN = 5 * ... (10회 반복)	5개 초과분 자동 삭제
MAXLEN 후	XLEN eventStream	약 5개만 남아있음 확인
XRANGE 확인	XRANGE eventStream - +	오래된 항목은 사라짐

MAXLEN ~ (근사치)가 MAXLEN = (정확치)보다 성능적으로 유리
(MAXLEN ~ (근사치)는 항목 수가 적을 때 거의 동작 안 함 — 실습에서는 'MAXLEN =' 사용)

실습 함정 포인트 — 자주 하는 실수

CLI 실습에서 자주 만나는 실수들

함정 1

> 대신 \$ 사용

XREADGROUP ... STREAMS s \$ → 에러. \$ 는 XGROUP CREATE 에만 사용.

✓ XREADGROUP에서는 > 사용

함정 2

Entry ID 잘못 복사

XPENDING에서 ID를 잘못 복사하면 XACK가 → 0 반환

✓ ID 형식: timestamp-seq 정확히 복사

함정 3

BUSYGROUP 에러

그룹이 이미 존재. DEL stream 후 재생성 권장.

✓ DEL eventStream → 재생성

함정 4

XACK 전 > 재호출

이미 읽은 메시지는 > 로 안 나옴 → 0 사용

✓ 재수신은 반드시 0

CLI 실습 vs 실제 코드 — 무엇이 다른가

CLI에서 한 것 → 코드에서 어떻게 구현하는가

CLI 명령	코드 구현	위치
XADD ...	<code>redis.xadd(stream, fields)</code>	<code>QueueService.enqueue()</code>
XGROUP CREATE ...	<code>redis.xgroup_create(stream, group)</code>	QueueService 초기화
XREADGROUP GROUP ...	<code>redis.xreadgroup(group, consumer, streams)</code>	EventConsumer 루프
XACK ...	<code>redis.xack(stream, group, id)</code>	EventConsumer 처리 후
XPENDING ...	<code>redis.xpending_range(stream, group)</code>	모니터링/DLQ 로직

S8에서 이 매핑을 코드로 구현 — QueueService 실제 연결

XINFO STREAM — 스트림 전체 정보 조회

XINFO STREAM eventStream — 스트림 종합 상태 확인

"length" → 전체 항목 수
"radix-tree-keys" → 내부 B트리 키 수
"last-generated-id" → 마지막 생성 ID
"max-deleted-entry-id" → MAXLEN으로 삭제된 마지막 ID
"entries-added" → 총 추가된 항목 수 (삭제 포함)
"groups" → Consumer Group 수
"first-entry" → 가장 오래된 항목
"last-entry" → 가장 최신 항목

$\text{entries-added} \geq \text{length}$ 이면 MAXLEN으로 삭제된 항목이 있다는 의미

운영 모니터링 시 groups × consumers × pending 조합으로 시스템 상태 판단

실습 디버깅 가이드 — 문제 발생 시 확인 순서

문제 상황별 디버깅 명령어

XREADGROUP >가 빈 결과

XLEN으로 항목 수 확인
XPENDING으로 PEL 확인
XINFO GROUPS에서 lag 확인

XACK 후에도 PEL에 남음

XACK 반환값 확인 (0이면 잘못된 ID)
XPENDING ID와 XACK ID 비교

BUSYGROUP 에러

XINFO GROUPS로 기존 그룹 확인
XGROUP DESTROY로 삭제 후 재생성

delivery_count 계속 증가

XPENDING으로 문제 메시지 ID 확인
처리 로직 오류 점검 → DLQ 이동 검토

Redis Streams 운영 베스트 프랙티스

교보 프로젝트에서 적용할 운영 원칙

MAXLEN 설정 필수

XADD 시 MAXLEN ~ 100000 항상 포함
메모리 무한 증가 방지

delivery_count 모니터링

≥ 3 이면 DLQ 이동 자동화
반복 실패 조기 감지

PEL 크기 주시

pending 값 지속 증가 = Consumer 처리 지연
Consumer 수 증가 검토

ACK 타임아웃 설정

60초 이상 미ACK = XAUTOCLAIM 대상
장애 Consumer 자동 재배포

그룹 분리 원칙

용도별 Group 분리 (NFT처리/감사/DLQ)
하나의 그룹에 모든 기능 몰아넣기 금지

S7

S7 전체 실습 흐름 복습

오늘 실습한 전체 시퀀스

✓ 환경 준비

Redis CLI 접속 + PING 확인 + DEL 클린업

✓ XADD 5개

이벤트 적재 + Entry ID 형식 확인

✓ XGROUP CREATE

nftConsumers 그룹 생성 + 초기화

✓ XREADGROUP >

5개 읽기 + PEL 등록 확인

✓ XACK 전체

PEL 비어있음 확인

✓ 미ACK 재수신

XACK 없이 종료 → 0 으로 재수신

✓ delivery_count

재배달 시 +1 증가 확인

✓ XAUTOCLAIM

오래된 미ACK 다른 Consumer로 재배분

✓ Consumer 2개 분배

6개 메시지 2 Consumer 중복 없이 분배

S8 미리보기 — QueueService 실제 구현

XADD → enqueue()

redis.xadd(STREAM_KEY, fields)
= WebhookReceiver가 호출하는 메서드

XGROUP CREATE → 초기화

redis.xgroup_create(STREAM_KEY, GROUP)
= QueueService 생성자에서 실행

XREADGROUP → read()

redis.xreadgroup(GROUP, consumer, streams)
= EventConsumer 루프 핵심

XACK → ack()

redis.xack(STREAM_KEY, GROUP, id)
= 처리 완료 후 반드시 호출

오늘 CLI에서 한 것 → S8에서 TypeScript 코드로 구현

S8: ioredis 라이브러리를 사용해서 위 4가지 메서드를 QueueService에 구현

핵심 용어 최종 정리 — S6+S7 통합

S6+S7에서 나온 용어 한 번에 정리

Stream

순서 있는 추가 전용 로그 자료구조

Entry ID

timestamp-seq 형식. XADD 반환값. 메시지 식별자.

Consumer Group

여러 Consumer가 하나의 스트림을 분담하는 구조

PEL

읽혔지만 XACK 안 된 메시지 목록

XACK

처리 완료 선언. PEL에서 제거.

delivery_count

재배달 횟수. ≥ 3 이면 DLQ 이동 대상.

XAUTOCLAIM

오랫동안 미ACK 메시지를 다른 Consumer로 재배포

At-least-once

최소 1번 처리 보장. PEL+XACK+재시작0 조합.

lag

아직 배달 안 된 메시지 수. XINFO GROUPS에서 확인.

COINCRAFT RESEARCH

정리

오늘 실습한 것 · 핵심 원리 확인 · 다음 세션 예고

S7 마무리

COINCRAFT
ACADEMY

S7 실습 요약 — 오늘 직접 확인한 것

손으로 타이핑해서 확인한 5가지

✓ XADD

이벤트 5개 추가 + Entry ID 형식 확인

✓ Consumer Group

XGROUP CREATE + XINFO GROUPS 확인

✓ XREADGROUP +
XACK

읽기 → PEL 등록 → ACK → PEL 제거 흐름

✓ 미ACK 재수신

XACK 없이 재시작 → 0으로 PEL 재수신 확인

✓ Consumer 2개 분배

6개 메시지를 2 Consumer가 중복 없이 분배

At-least-once — 오늘 실습으로 직접 확인한 원리

이론(S6)을 실습(S7)으로 확인 — 이제 진짜 이해한 것

1. 메시지 유실 없음

XACK 없이 종료해도 PEL에 살아있음
재시작 후 0으로 재수신 확인

2. 중복 방지

동일 Consumer Group 내
동일 메시지는 하나의 Consumer만 받음

3. 재처리 가능

delivery_count 증가 확인
DLQ 기준(≥ 3)도 직접 확인

4. 수평 확장

Consumer 2개 → 메시지 자동 분배
코드 변경 없음

핵심 명령어 최종 암기

이 명령어들을 외우면 Redis Streams 운영 가능

XADD stream * f v	스트림 추가	반환: Entry ID
XLEN stream	항목 수	반환: 정수
XRANGE stream - +	전체 조회	반환: 항목 목록
XGROUP CREATE s g \$ MK	그룹 생성	반환: OK
XREADGROUP GROUP g c COUNT N STREAMS s >	새 메시지 읽기	반환: 항목 목록
XREADGROUP GROUP g c COUNT N STREAMS s 0	PEL 재수신	반환: PEL 항목
XACK s g id	처리 완료	반환: ACK된 수
XPENDING s g - + N	PEL 조회	반환: PEL 목록

S7 완료 기준

이것을 할 수 있으면 S7 완료

✓	Streams 실습	XADD → XLEN → XRANGE 직접 실행 완료
✓	Consumer Group	XGROUP CREATE + XREADGROUP + XACK 흐름 완료
✓	미ACK 재수신	XACK 없이 종료 → 0으로 재수신 확인 완료
✓	delivery_count	재배달 시 delivery_count 증가 확인 완료
✓	Consumer 2개 분배	2개 Consumer 메시지 분배 중복 없음 확인 완료

다음 세션 (S8): Webhook 보안 검증(HMAC) + QueueService 실제 구현

S7

다음 세션 — S8 Webhook 보안 + QueueService 구현

S7 완료 = Redis 손에 익음 → S8: 실제 코드에 연결

HMAC-SHA256 검증

VASP 서명 헤더 검증
서명 없는 요청 거부

QueueService 구현

enqueue() = XADD 호출
Consumer Group 초기화

receiveWebhook 완성

HMAC 검증 → enqueue() → 202
S5 stub → 실제 구현

통합 테스트

서명 검증 + Queue 적재
E2E 흐름 검증

S8 완료 = WebhookReceiver + QueueService 완전 연결 → 실제 VASP 이벤트 처리 가능

S7

수고하셨습니다

S7 — Redis Streams CLI 실습 At-least-once 재처리 시뮬레이션

오늘 직접 확인한 것

XADD/XREADGROUP/XACK ✓
미ACK 재수신 ✓
Consumer 2개 분배 ✓

다음 세션 (S8)

HMAC 서명 검증
QueueService 실제 구현
receiveWebhook 완성

질문 · 피드백 환영합니다